Instalación y Configuración de un ERP

Unidad 3

Introducción 3
Contexto histórico de los ERP-CRM 3
Sistemas ERP-CRM en la propia empresa 4
Sistemas ERP-CRM en la nube 4
¿Qué elegir? ¿Sistema en la propia empresa o en la nube? 5
¿Y el software para nuestro sistema ERP? 5
Tipos de instalación de un sistema ERP 6
Preparación del servicio para instalar el sistema ERP 8
Instalación de un sistema ERP Odoo 10
Odoo 17 en Ubuntu Server 11
Odoo 17 en Docker: 12
- Parte 1: Instalar Docker Desktop 12
- Parte 2: Contenedor Odoo en producción 14
- Parte 3: Contenedor Odoo para desarrollo 16
- Parte 4: Docker Compose para Odoo (RECOMENDADA) 18

Odoo 18 en nuestro Docker Desktop: 21

Unidad 3

Introducción

A continuación comentaremos los factores generales que debemos tener en cuenta para la instalación de un sistema ERP, además terminaremos hablando de las distintas posibilidades que tenemos a la hora de instalar "Odoo", el software que utilizaremos durante este curso.

Contexto histórico de los ERP-CRM

Las posibilidades para la instalación de un sistema ERP-CRM se han multiplicado en los últimos años. Haciendo un breve repaso histórico podemos reflexionar sobre los motivos técnicos y económicos de esta **evolución**:

- En los primeros años de la informática empresarial, el Mainframe era casi la única opción: un único ordenador muy grande y caro con varias terminales. Este Mainframe era instalado en el edificio de la empresa. Esta solución solo estaba al alcance de grandes empresas.
- La llegada de los computadores personales y miniordenadores hizo que muchas empresas más pequeñas utilizaran esa solución.
    - En ocasiones era un único ordenador pequeño gestionado por la empresa.
    - Mientras tanto, los mainframes seguían ocupando el espacio de las grandes empresas.
    - En sus inicios, el PC y similares tenían carencias tanto a nivel de potencia y como de sistemas operativos orientados al mercado doméstico y no al mercado profesional. Aun así, su aterrizaje en la vida cotidiana influenció a la sociedad y a la industria, llevando que fabricaran servidores con la arquitectura de los PC y similares, pero con una mayor fiabilidad requerida en el contexto industrial.
        - Esto es una gran mejora que permanece hasta hoy en día. Los servidores con esta arquitectura mejoraron considerablemente el hardware con procesadores y memorias más estables y sistemas operativos libres y propietarios más robustos (Unix, Linux, Windows NT/Server...).
- La llegada de Internet supuso otra alternativa. Ahora las empresas podían usar una red común para interconectar sus sedes, tanto con mainframes como con servidores PC, se podía tener un programa centralizado.
    - Al mismo tiempo, algunas empresas pudieron ofrecer servicios por Internet a otras empresas. Este fue el inicio de lo que llamamos "servicios en la nube".

Sistemas ERP-CRM en la propia empresa

La opción de tener un servidor en la propia empresa tiene una serie de ventajas e **inconvenientes** (relacionados entre sí) y cuyos principales problemas vienen dados tanto por los **costes económicos** como por la **seguridad de los datos**.

Tener un servidor en la propia empresa con un sistema ERP-CRM (o con cualquier otro tipo de servicio) supone algunos **retos**:

- Poner en marcha un servicio requiere una inversión inicial en hardware.
- El escalado de hardware para aumentar potencia/disminuir potencia es problemático:
  - Habitualmente se tiene hardware infrautilizado.
  - El aumento de potencia de un servicio requiere escalado vertical (aumento de prestaciones del servidor) o escalado horizontal (comprar más equipos).
    - El hardware limita las posibilidades de cómputo, así que es imposible escalar para aumentar potencia en momentos puntuales.
- El mantenimiento del sistema informático, el suministro de energía y la seguridad del sistema son gastos y responsabilidades asociados a la empresa.
  - Una ventaja de la gestión de datos interna es que no están en ordenadores fuera de la empresa, evitando riesgos tales como el espionaje industrial.

Sistemas ERP-CRM en la nube

El gasto tanto energético como de potencia de computación en una sola empresa suele ser desigual durante el día, suponiendo esto una ineficiencia energética. Por este motivo, algunas empresas, especialmente aquellas que ofrecen servicios por Internet, se plantearon vender esa potencia cuando no la necesitaban o compartirla entre muchos clientes. Esto propició el nacimiento de lo que hoy llamamos "**los servicios en la nube**".

Con la llegada de los "servicios en la nube" (que realmente son servidores de otra empresa) las empresas pagan una cuota (fija, por tiempo de cómputo, por uso, etc.), pero a cambio:

- No realizan gastos relacionados con el hardware (instalación y escalado).
- No realizan gastos relacionados con el consumo eléctrico.
- Se reducen de forma drástica los gastos en mantenimiento y seguridad.
- Facilitan el acceso: para operar con estos sistemas la empresa solo necesita un dispositivo (ordenador personal, smartphone, etc.) con conexión a Internet.
- Si se necesita más potencia, solo hay que contratarla (escalado vertical/horizontal).

Aun así, el uso de "servicios en la nube" poseen varios **inconvenientes**:

- En algunos contextos, puede resultar más **caro** que poner en marcha tu la infraestructura.
- Los **datos** están almacenados físicamente **en un servidor de otra empresa**, con posibles problemas relacionados (por ejemplo, espionaje industrial).
- Para el uso del sistema **dependen** tanto de una **buena conexión a Internet** como del buen **funcionamiento general de la empresa que presta servicios**.

¿Qué elegir? ¿Sistema en la propia empresa o en la nube?

No existe una respuesta "tajante" a esta pregunta, ya que depende del contexto y es algo a estudiar concienzudamente antes de implantar un sistema.

Hoy en día en los entornos empresariales conviven ambas opciones mencionadas, e incluso opciones híbridas (servidor en la empresa, pero apoyo puntual o algunos servicios en la nube).

Los principales factores a la hora de tomar esta decisión son:

- **El cumplimiento de las leyes de protección de datos.** Este punto sobre todo influye en la decisión de contratar o no un "servicio en la nube", ya que guardando determinados datos en determinados servicios en la nube podríamos estar incumpliendo la ley.
  - En Europa existe el reglamento RGPD y en España la legislación vigente en materia de protección de datos viene definida por la LOPDGDD.
- **El precio de la electricidad y el consumo del hardware**.
  - Existen dispositivos hardware orientados a tener un bajo consumo.
- **El precio del hardware.**
  - Existen dispositivos orientados a tareas de servidor muy baratos.

¿Y el software para nuestro sistema ERP?

Durante la introducción hemos hablado de costes relacionados con el sistema que alojaría nuestro ERP. Pero… ¿Y el software?. Existen multitud de opciones software ERP, tanto gratuitos, de pago, libres, mixtos (partes libres, partes de pago), etc. Existe un software ERP llamado **Odoo**, que se presenta en dos versiones: "**Community Edition**" (software libre y gratuito) y "**Enterprise Edition**" (de pago). En este curso utilizaremos **Odoo "Community edition",** ya que es libre y gratuito. Por simplicidad, cuando nos refiramos a Odoo, estaremos refiriéndonos a esta versión. Más información en https://www.odoo.com/es_ES/ y https://github.com/odoo/odoo.

Tipos de instalación de un sistema ERP

En este apartado vamos a tratar las distintas maneras en la que se puede instalar un sistema ERP y a mencionar los pros y contras de cada tipo de solución.

Comenzaremos con las soluciones **no recomendadas** ("cutres"), para pasar aquellas simples pero correctas e ir evolucionando hacia las más sofisticadas:

- **La carpeta compartida:** se trata de una práctica cada vez menos usada pero que se mantiene en muchas pequeñas empresas.
  - Hace años, algunos programas de gestión estaban pensados para un solo usuario. La única opción que daban para poder ser accedidos por varios ordenadores en una red local era compartir la carpeta de la base de datos y configurar varias instalaciones para acceder al mismo archivo. Además, esa "base de datos" era un fichero de "Access" o ficheros de texto plano.
  - **Esta solución no es correcta ni recomendable**: es una solución que propone muchos problemas, tanto con el acceso simultáneo, como problemas tanto de seguridad como de integridad.
- Instalación **On-Premise**: aquí entra en juego un servidor instalado en la propia empresa.
  - Los ordenadores cliente pueden acceder al sistema ERP mediante un software cliente del propio ERP o incluso mediante un navegador web.
    - Se debe configurar correctamente la red del sistema tanto para evitar conexiones externas no permitidas, como para proporcionar acceso seguro (si se requiere) desde fuera de la red.
    - Si se necesita acceder desde fuera de la red local, hay que configurar correctamente la red y la seguridad del acceso externo. Se suele pagar por el software entero o la instalación y el mantenimiento.
  - Si el servidor funciona de una forma segura y esta opción es viable económicamente para la empresa, es una solución correcta.
- Instalación como servicio **en la nube**: en esta solución prescinde de servidor en la empresa y se subcontrata la computación. Simplifica la instalación y el acceso externo. Además, se paga por lo que se necesita y es fácilmente escalable. Los "servicios en la nube" puede ser de muchos tipos, algunos ya comentados en la unidad 2, pero se suele distinguir entre:
  - **IAAS** (Infraestructura como servicio): nos proporcionan acceso a servidores, máquinas virtuales o contenedores. Hay que instalar y segurizar el sistema operativo, el ERP y todo lo demás. Puede que la empresa de la nube ya nos lo

ofrezca con un sistema operativo preinstalado, pero deberíamos tener control total de ese sistema operativo.

  - Ejemplo: los **VPS** (Virtual Private Server / Servidor Privado Virtual) entran dentro de este tipo de nube.

- **PAAS** (Plataforma como servicio): En esta nube ya está el sistema operativo y algunos programas configurados. Sobre este sistema se puede desplegar el sistema ERP.
  - Ejemplo: lo que consideramos como **hosting tradicional** (PHP, MySQL, etc.).
- **SAAS** (Software como servicio): en este caso ya está el ERP instalado y nos dan acceso a determinadas características según contratemos (cantidad de usuarios, acceso simultáneo, espacio de almacenamiento, copia de seguridad, etc.). No hay que preocuparse de nada más, pero estás limitado al programa que el proveedor ofrece.
  - Ejemplo: soluciones tipo Gmail o Google Drive, donde tenemos todo puesto en marcha y solo consumimos el servicio.
- Aparte de estos tipos de "servicios en la nube", podemos tener algunos servicios de forma individual en la nube, como por ejemplo servicios de bases de datos (como Firebase) o servicios de APIs REST o GRAPHQL, etc. En cualquier caso, deben garantizar una alta disponibilidad, seguridad y escalabilidad.

Preparación del servicio para instalar el sistema ERP

Cada sistema ERP es distinto y posee necesidades de potencia distintas. Estas necesidades suelen depender de factores como:

- Cantidad de datos que alberga el sistema.
- Número de usuarios simultáneos
- Tecnología utilizada por el software ERP.
- Sistema operativo sobre el que corre el software ERP.

Para poner en marcha un sistema ERP es recomendable utilizar hardware diseñado para servidores. La calidad de los componentes, la refrigeración o tecnologías como el ECC en la memoria RAM hacen más estables estos ordenadores frente a PC domésticos u otras alternativas para el usuario final (Raspberry Pi, Arduino, etc.).

Los usuarios que interactúan con el sistema ERP, sí pueden usar PCs, tablets, smartphones u otros sistemas domésticos como cliente del servicio.

Otro aspecto a tener en cuenta es la seguridad, teniendo en cuenta tanto medidas de seguridad pasiva como activa. Deben tenerse en cuenta aspectos como:

- Protección contra personas (ataques físicos y remotos).
- Protección contra los elementos: calor, humedad, sobretensiones o incendios.
- Disponibilidad del sistema y copia de seguridad.
- Cumplimiento de la legislación vigente.

No es tarea de este curso explorar todas las protecciones disponibles, pero deben tenerse en cuenta y exigirlas en una instalación real. Algunas de las medidas de protección básicas son:

- El servidor debe tener un SAI que lo proteja y lo mantenga encendido en caso de fallo de la red eléctrica.
- Es recomendable que el servidor en sí disponga de elementos (procesador, RAM, discos, etc.) por encima de la potencia mínima necesaria para que funcione el sistema ERP.
    - ➢ No es necesario pasarse demasiado, ya que si lo hacemos incrementaremos tantos gastos de hardware y el consumo eléctrico sin obtener un gran

beneficio. Normalmente, estos programas tienen una documentación en la que describen los requerimientos mínimos.

- Respecto a la seguridad de los datos, se recomienda redundancia en los discos, ya sea con RAID o con sistemas de archivos redundantes como ZFS o BTRFS.
  - Esta redundancia no excluye la necesidad de establecer una política de copias de seguridad externas al sistema.

Una vez puesto en marcha el servidor hay que elegir el sistema operativo base y un posible sistema de virtualización. En el caso del sistema ERP Odoo, que tratamos en estos apuntes, para una puesta en marcha en un sistema real se recomienda Ubuntu Server.

Este sistema operativo puede ser el sistema instalado en la máquina o estar virtualizado.

La virtualización podemos realizarla con:

- Máquinas virtuales con hipervisor (tipo Virtualbox). Estas máquinas realizan una simulación completa del hardware, influyendo esto en una disminución de rendimiento.
- Una alternativa a la virtualización con hipervisor, pero con mejor rendimiento (cercano al rendimiento nativo) son los contenedores, ya sean completos como LXD o contenedores de aplicaciones como **Docker**.

Sistemas operativos base como Proxmox simplifican la gestión tanto de máquinas virtuales como de contenedores, copias de seguridad de los mismos y almacenamiento en red.

En caso de optar por la nube, si contratamos un IAAS (VPS) también debemos tener en cuenta la potencia contratada. De hecho, es más importante afinar correctamente, ya que podemos incrementar los costes sin tener un beneficio en cuanto a rendimiento.

Instalación de un sistema ERP Odoo

En este apartado en primer lugar trataremos los requisitos mínimos para instalar el sistema ERP **Odoo** en su versión 17. Tras ello, explicaremos cómo realizar una instalación manual en un sistema Ubuntu. Finalmente, explicaremos cómo poner en marcha Odoo 17 mediante contenedores Docker usando Docker y Docker Compose.

A efectos prácticos, Odoo 17 no necesita mucha potencia para funcionar. Puede funcionar sin problemas en cualquier ordenador con varios núcleos y al menos 512MB de RAM, aunque con esa configuración, si recibe muchos accesos simultáneos, la máquina se puede quedar corta.

Como en todas las aplicaciones que consultan bases de datos, el acceso al disco puede suponer un cuello de botella. Por eso es recomendable utilizar unidades SSD, RAIDs o sistemas de archivos como ZFS o BTRFS con varios discos.

Odoo 17 funciona perfectamente en máquinas virtuales y contenedores. Algunas opciones de configuración pueden ser:

- Sistema operativo: Ubuntu Server e instalación directa de Odoo.
- Sistema operativo: Ubuntu Server, Virtualización con KVM o similar. Las máquinas virtuales tendrán instalado Ubuntu Server.
- Sistema operativo: Ubuntu Server, contenedores con LXD de Ubuntu.
- Sistema operativo: Ubuntu Server, contenedores Docker.
- Sistema operativo: Proxmox, máquinas virtuales o contenedores LXC gestionados por Proxmox.

Como se ve, en todas las ocasiones se eligen máquinas reales y virtuales Ubuntu. Esto es porque Odoo está desarrollado en ese sistema y esto nos ayuda a garantizar que la implantación del sistema funcione correctamente.

Odoo 17 en Ubuntu Server

Para realizar una instalación manual de Odoo, recomendamos seguir los pasos indicados en su documentación oficial, disponible en:

https://www.odoo.com/documentation/17.0/index.html

https://www.odoo.com/documentation/17.0/administration/install.html

Odoo 17 en Docker:

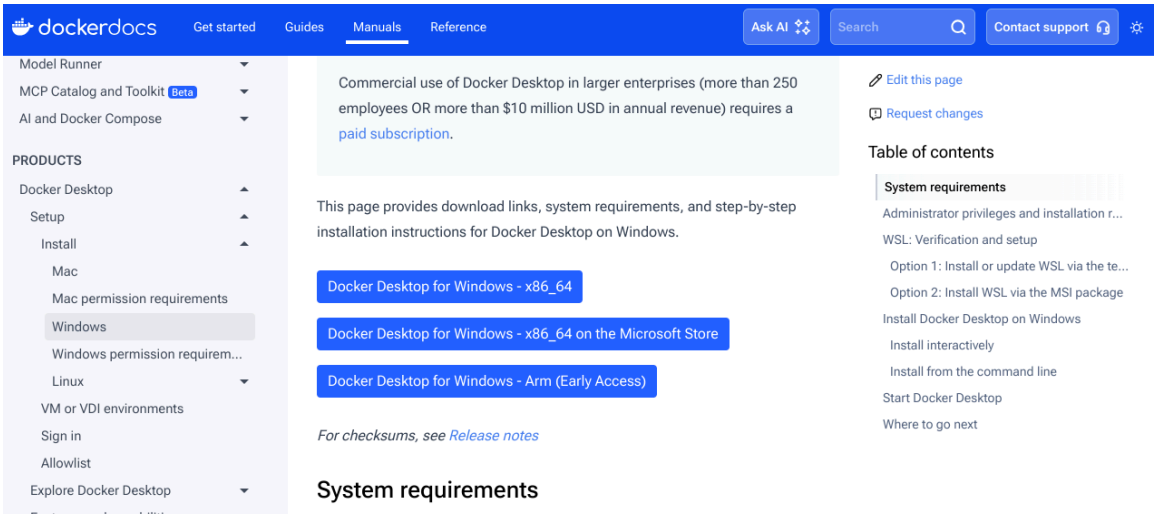
- ### Parte 1: Instalar Docker Desktop

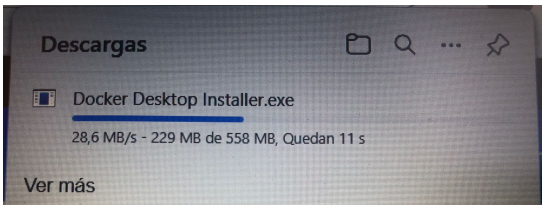
Si no conocéis como usar “Docker”, puede seros muy útil revisar el curso con ejemplos prácticos https://sergarb1.github.io/CursoIntroduccionADocker/

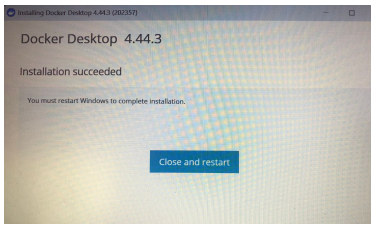
Si ya conoces como usar “Docker”, puede seros muy útil esta “Hoja de trucos” https://raw.githubusercontent.com/sergarb1/CursoIntroduccionADocker/main/FuentesCurso/Docker%20CheatSheet%20COMPLETA.pdf

EJERCICIO 1:

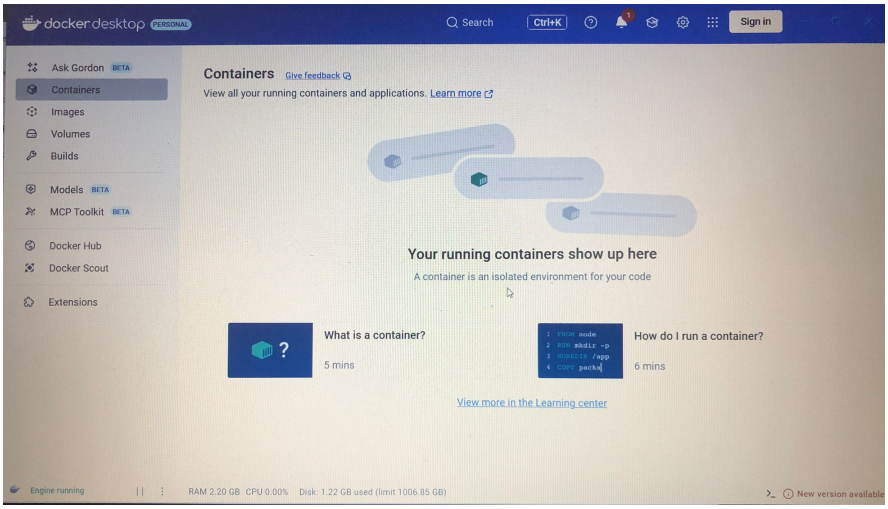
a) Antes de nada como trabajaremos con sistema Windows vamos a instalar Docker Desktop en nuestro ordenador. Para ellos hacemos clic en este enlace https://docs.docker.com/desktop/setup/install/windows-install/ e instalamos “Docker Desktop for Windows - x86_64“. Sigue los pasos e instala Dockers Destkop en tu PC.







b) Una vez instalado Docker Desktop lo vamos a abrir para ver que de momento no tenemos creado ningún contenedor dentro.



- **Parte 2: Contenedor Odoo en producción**

Para poner en marcha Odoo 17 en modo producción crearemos dos contenedores:

- El primer contenedor contendrá la base de datos PostgreSQL en su versión 15.
- El segundo contenedor contendrá el servidor Odoo.

Creamos el contenedor de PostgreSQL con:

```
docker run -d -v /home/usuario/OdooDesarrollo/dataPG:/var/lib/postgresql/data -e POSTG
RES_USER=odoo -e POSTGRES_PASSWORD=odoo -e POSTGRES_DB=postgres --name db postgres:15
```

Donde los parámetros indican lo siguiente:

- **"-d"** : ejecuta el contenedor en segundo plano.
- **"-v /home/usuario/OdooDesarrollo/dataPG:/var/lib/postgresql/data"**: monta el directorio del contenedor "/var/lib/postgresql/data" (donde se encuentra la información almacenada por PostgreSQL) en el directorio del anfitrión "/home/usuario/OdooDesarrollo/dataPG".
  - El fin de esto es almacenar la información de la base de datos en la máquina anfitriona.
- **"-e POSTGRES_USER=odoo -e POSTGRES_PASSWORD=odoo -e POSTGRES_DB=postgres"**: establece dentro del contenedor esas variables de entorno. A efectos prácticos, esas variables le indican que creen en la base de datos un usuario "odoo", con contraseña "odoo" y que la base de datos a usar se llama "postgres".
- **"--name db"** : nombre que le daremos a nuestro contenedor Docker.
- **"postgres:15"**: indicamos que usaremos la imagen de Docker Hub llamada "postgres" y de entre ellas usaremos la versión 15.
  - Más información de esta imagen en https://hub.docker.com/_/postgres

Importante: si en lugar del parámetro "-d", utilizamos el parámetro "-t", ejecutaremos el contenedor en primer plano y veremos en la terminal información del inicio de PostgreSQL u Odoo. Esto es interesante para detectar problemas.

Con el contenedor PostgreSQL ya en marcha, creamos el contenedor con Odoo con:

```
docker run -d -v -p 8069:8069 --name odooprod --user="root" --link  db:db odoo:17
```

Donde los parámetros indican lo siguiente:

- **"-d"** : ejecuta el contenedor en segundo plano.

- **"-p 8069:8069"**: mapeamos el puerto 8069 del contenedor (donde accedemos a Odoo) al puerto 8069 de la máquina anfitrión, para poder acceder a Odoo con http://localhost:8069
- **"--name odooprod"** : damos el nombre "odooprod" a nuestro contenedor.
- **"--user="root""**: fuerza a que el contenedor se ejecute internamente como "root" y no como el usuario "odoo" que va por defecto en la imágen.
- **"--link db:db"** : enlazamos con una red virtual este contenedor con el contenedor "db" .

- **Parte 3: Contenedor Odoo para desarrollo**

Para lanzar Odoo en un contenedor preparado para desarrollo, creamos también dos contenedores.

Creamos el contenedor de PostgreSQL de forma similar a como hicimos en el apartado anterior con:

```
docker run -d -v /home/usuario/OdooDesarrollo/dataPG:/var/lib/postgresql/data -e POSTG
RES_USER=odoo -e POSTGRES_PASSWORD=odoo -e POSTGRES_DB=postgres --name db postgres:15
```

Un servidor de producción está pensado para ponerlo en marcha, realizar pocas paradas y mantener el contenido del contenedor. Se suele hacer copias completas del contenido para únicamente restaurar el contenedor en caso de urgencia.

Sin embargo, en entornos de desarrollo, es más habitual "romper cosas". A efectos prácticos, es habitual reiniciar contenedores o incluso borrarlos y construirlos de nuevo.

Para ello, vamos a modificar la forma de crear los contenedores guardando algunas cosas en volúmenes para realizar "persistencia" del servicio Odoo.

Creamos el contenedor de Odoo, con algunas diferencias respecto al anterior:

```
docker run -d -v /home/usuario/OdooDesarrollo/volumesOdoo/addons:/mnt/extra-addons -v
/home/usuario/OdooDesarrollo/volumesOdoo/firestore:/var/lib/odoo/filestore -v /home/us
uario/OdooDesarrollo/volumesOdoo/sessions:/var/lib/odoo/sessions -p 8069:8069 --name o
doodev --user="root" --link db:db -t odoo:17 --dev=all
```

A continuación, comentamos las diferencias:

- "**-v /home/usuario/OdooDesarrollo/addons:/mnt/extra-addons**": la imagen de Odoo 17 por defecto carga los módulos del directorio del contenedor "/mnt/extra-addons", por lo cual mapeamos ese directorio a nuestro directorio de la máquina anfitrión "/home/usuario/OdooDesarrollo/addons", donde desarrollaremos usando un IDE externo.
- **"-v /home/usuario/OdooDesarrollo/volumesOdoo/firestore:/var/lib/odoo/filestore**
- **" y "-v /home/usuario/OdooDesarrollo/volumesOdoo/sessions:/var/lib/odoo/sessions"**: como en desarrollo es posible que paremos y montemos muchas veces los contenedores Docker, montamos estos volúmenes para tener persistencia de los directorios de Odoo "firestore" y la "sessions". Para ello, mapeamos esos directorios del contenedor a nuestra máquina anfitrión dentro del directorio "/home/usuario/OdooDesarrollo/volumesOdoo/".
- **" --dev=all"**: le pasa ese parámetro a Odoo para facilitar tareas de desarrollo.

- El detalle de que realiza esta opción se puede ver en el siguiente enlace https://www.odoo.com/documentation/17.0/es/developer/reference/cli.html

Atención: para poder desarrollar sin problemas, es recomendable darle todos los permisos al directorio "/home/usuario/OdooDesarrollo/volumesOdoo/addons", con un comando similar a "sudo chmod -R 777 /home/usuario/volumesOdoo/addons".

Con esto, tendremos listo nuestro entorno de desarrollo "Dockerizado". Hemos conseguido que los contenedores corran de manera aislada los servicios de base de datos y Odoo, mientras que nosotros podremos desarrollar usando un IDE instalado en el anfitrión trabajando dentro del directorio **"/home/usuario/OdooDesarrollo/addons".**

- **Parte 4: Docker Compose para Odoo (RECOMENDADA)**

Docker Compose es una herramienta que nos facilita el despliegue de varios contenedores usando una configuración definida en un fichero. Este fichero por defecto debe llamarse "docker-compose.yml".

Si nos situamos en el directorio donde esté nuestro fichero "docker-compose.yml", podemos iniciar el servicio completo simplemente escribiendo:

```
docker compose up –d
```

Al lanzar esta orden, se crearán en el directorio actual:

- Carpeta "addons": ahí desarrollaremos nuestros módulos mediante un IDE externo.
- Carpeta "pgData": ahí se almacenará la persistencia de datos de nuestra base de datos.

Atención: para poder desarrollar sin problemas, es recomendable darle todos los permisos al directorio "addons", con un comando similar a "sudo chmod -R 777 ./addons".

Podemos parar el servicio completo simplemente escribiendo:

```
docker compose down
```

Adjuntamos a esta unidad un zip con el fichero "docker-compose.yml" para entorno de producción y con el fichero "docker-compose.yml" para entorno de desarrollo. A continuación, además mostramos el contenido del fichero "docker-compose.yml" para entorno de desarrollo.

Fichero "docker-compose.yml" (desarrollo):

```
version: '3.3'


services:
#Definimos el servicio Web, en este caso Odoo
  web:
```

```
      #Indicamos que imagen de Docker Hub utilizaremos
      image: odoo:17
      #Indicamos que depende de "db", por lo cual debe ser procesada primero "db"
      depends_on:
      - db


      # Port Mapping: indicamos que el puerto 8069 del contenedor se mapeara con el mi
smo puerto en el anfritrion
      # Permitiendo acceder a Odoo mediante http://localhost:8069
      ports:
      - 8069:8069


      # Mapeamos el directorio de los contenedores (como por ejemplo"/mnt/extra-addons
" )
      # en un directorio local (como por ejemplo en un directorio"./volumesOdoo/addons
")
      # situado en el lugar donde ejecutemos "Docker compose"
      volumes:
      - ./volumesOdoo/addons:/mnt/extra-addons
      - ./volumesOdoo/odoo/filestore:/var/lib/odoo/filestore
      - ./volumesOdoo/odoo/sessions:/var/lib/odoo/sessions


      # Definimos variables de entorno de Odoo
      environment:
      - HOST=db
      - USER=odoo
      - PASSWORD=odoo
      # Indica que pasa ese parametro al arrancar el servicio Odoo
      command: --dev=all
#Definimos el servicio de la base de datos
  db:
      image: postgres:15
      # Definimos variables de entorno de PostgreSQL
```

```
      environment:
      - POSTGRES_PASSWORD=odoo
      - POSTGRES_USER=odoo
      - POSTGRES_DB=postgres
      # Mapeamos el directorio del contenedor "var/lib/postgresql/data" en un director
io "./volumesOdoo/dataPostgreSQL"
      # situado en el lugar donde ejecutemos "Docker compose"
      volumes:
      - ./volumesOdoo/dataPostgreSQL:/var/lib/postgresql/data
```

Puesta en marcha de Odoo 17

Una vez realizada la instalación con cualquiera de las alternativas propuestas anteriormente, accederemos mediante nuestro navegador a Odoo mediante http://localhost:8069 y deberemos realizar una configuración inicial. Aquí un ejemplo de dicha configuración:

A primera vista, se nos mostrará un “Password maestro” que podemos cambiar si queremos. Deberemos almacenar en un lugar seguro ese “Password maestro” para poder recuperar nuestro sistema ante problemas de usuarios.

Atención: si perdemos el “Password maestro”, podemos ponerlo en blanco editando “/etc/odoo/odoo.conf” (o el fichero “.odoorc” dentro del “home”) y eliminando (o poner un comentario con #) el campo “admin_password”. Si lo hacemos así y reiniciamos el servicio, Odoo nos dirá que no hay “Password maestro” y nos sugerirá que creemos un nuevo password.

Además, se nos pedirá configurar Odoo según los parámetros de nuestra instalación. En esta configuración crearemos un usuario administrador, e indicaremos nuestro país (esto realizará algunas adaptaciones para empresas locales) e idioma de Odoo. Incluso nos permite cargar la instalación con datos de demostración (útil para realizar pruebas, conocer cómo funciona Odoo, etc.).

Una vez esté todo listo, al pulsar “Create database” se creará inicializará Odoo. Tened paciencia (tarda unos minutos).

Esto indica que Odoo 17 se ha instalado correctamente y ya podemos trabajar con él.

Odoo 18 en nuestro Docker Desktop:

Creación de los contenedores de Odoo 18 y PostgreSQL utilizando unos contenedores ya creados en este repositorio de GitHub https://github.com/vjmoreno/odoo-18-docker.

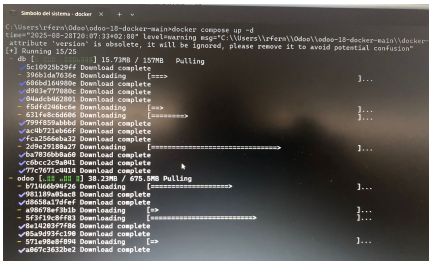
EJERICICIO 2

a) Accede al repositorio anterior de github y descarga el código en tu PC (odoo-18-docker-main.zip) en una carpeta/directorio que será la carpeta de tu proyecto.

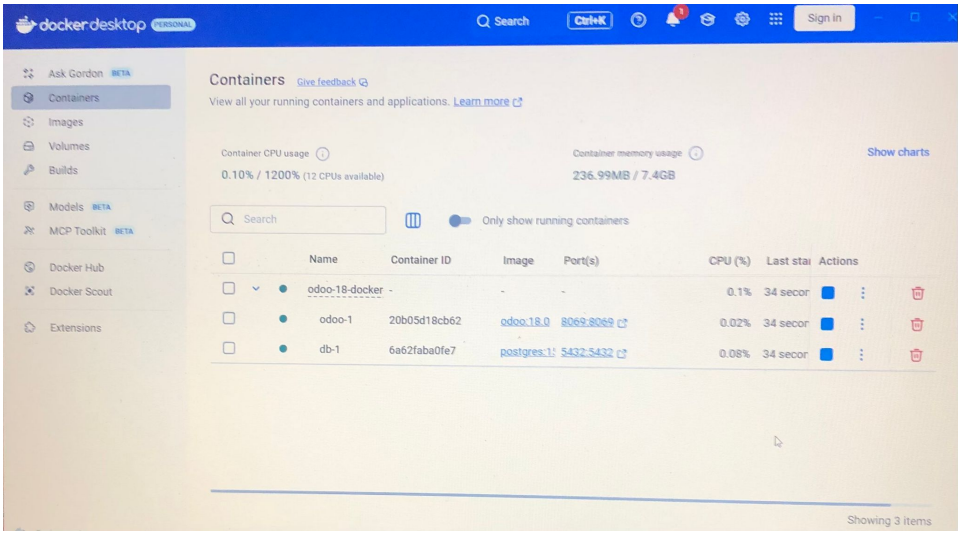
b) Ingresa a la carpeta del proyecto y levanta los "Contenedores" como se indica en el Readme del repositorio. Para iniciar los contenedores de Odoo y PostgreSQL, ejecuta esta instrucción en la Consola/Terminal de tu PC:

```
./run.sh
```

Esto iniciará los contenedores en segundo plano.

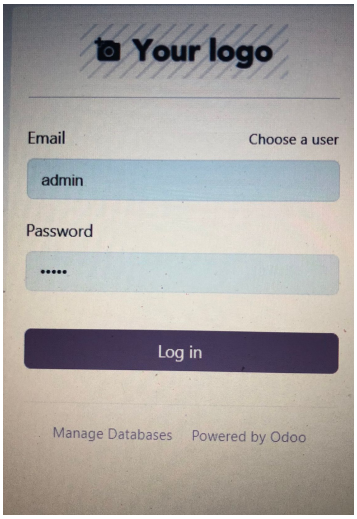


Si ahora accedemos a Docker Desktop veremos que ya se muestran los contenedores iniciados, tanto el de Odoo como el de PostgreSQL.



Puesta en marcha de Odoo 18

Una vez realizada la instalación con cualquiera de las alternativas propuestas anteriormente, accederemos mediante nuestro navegador a Odoo mediante http://localhost:8069 , se mostrará la siguiente pantalla:



Nos logearemos con Email (admin) y Password (admin) y accederemos a nuestro Sistema ERP Odoo. Mostrándose la siguiente pantalla:

